DragonBoard

使用指南

版本号: 2.3

发布日期: 2025.08.14

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2019.12.31	AWA1637	初始版本
1.1	2020.05.16	AWA1730	修改一些排版问题
1.2	2022.07.16	AWA1960	1. 修复了一些 Bug; 2. 新增了霍尔传感器、PMU、电源按键等测试用例; 3. 修改了一些测试用例的描述; 4. 更新 DragonBoard 版本号为 V3.0
1.3	2022.10.09	AWA1696	触摸屏测试支持手动和半自动测试
1.4	2022.11.03	AWA1960	增加了适用范围
1.5	2023.03.09	AWA1960	1. 增加了适用范围; 2. 更换了 UI 界面图片; 3. 新增了摄像头闪光灯的测试用例
1.6	2023.04.26	AWA1696	增加了 TP 自动化测试用例
1.7	2023.11.13	AWA1960	修订文档，更新了部分内容
1.8	2024.11.20	AWA1960	1. 增加了适用范围; 2. 新增了 UFS、USB3.0 和 USB 数字耳机测试用例
1.9	2025.03.07	AWA1960	1. 增加了适用范围; 2. 新增了光线传感器测试用例; 3. 新增了 FAQ 内容
2.0	2025.04.02	AWA1960	修改编译二合一固件的方法，区分 GRF 版本与非 GRF 版本
2.1	2025.07.14	AWA1960	修改音频测试时外接的耳机类型说明
2.2	2025.07.25	AWA1960	1. 增加了 GPS 测试用例; 2. 修改了文档中一些编辑错误
2.3	2025.08.14	AWS0782	1. 增加了适用范围; 2. 修改了摄像头、音频测试用例的配置参数

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1

2 系统介绍 **2**
2.1 系统简介 2
2.2 源码结构介绍 2
2.3 系统配置与编译 3
2.3.1 配置 DragonBoard 4
2.3.2 编译 DragonBoard 5
2.3.3 生成固件签名密钥 5
2.3.4 打包 DragonBoard 5
2.3.5 烧写 DragonBoard 6
2.3.6 编译及验证 Android & DragonBoard 二合一固件注意事项 6
2.3.6.1 编译 Android & DragonBoard 二合一固件 6
2.3.6.2 非 GRF 版本下编译 Android & DragonBoard 二合一固件 6
2.3.6.3 GRF 版本下编译 Android & DragonBoard 二合一固件 8
2.3.6.4 验证 Android & DragonBoard 二合一固件 10

3 测试流程 **11**
3.1 PMU 测试 14
3.2 DRAM 测试 14
3.3 RTC 测试 14
3.4 eMMC 闪存测试 14
3.5 NAND 闪存测试 14
3.6 UFS 闪存测试 14
3.7 SD 卡测试 14
3.8 重力传感器测试 15
3.9 霍尔传感器测试 15
3.10 光线传感器测试 15
3.11 GPS 测试 15
3.12 触摸屏测试 15
3.13 按键测试 16
3.14 电源按键测试 16
3.15 Wi-Fi 测试 16
3.16 蓝牙测试 16
3.17 以太网测试 17
3.18 摄像头测试 17

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.19 闪光灯测试 17
3.20 音频模块测试 17
3.21 U 盘测试 18
3.22 USB 3.0 测试 18
3.23 USB_HOST 测试 18
3.24 USB_OTG 测试 19
3.25 USB 数字耳机测试 19
3.26 HDMI 输出测试 19
3.27 CVBS 输出测试 19

4 配置文件 20
4.1 模块配置示例 20
4.2 PMU 测试配置 21
4.3 DRAM 测试配置 21
4.4 RTC 测试配置 21
4.5 eMMC 测试配置 22
4.6 NAND 测试配置 22
4.7 UFS 测试配置 22
4.8 SD 卡测试配置 23
4.9 重力传感器测试配置 23
4.10 霍尔传感器测试配置 23
4.11 光线传感器测试配置 24
4.12 GPS 测试配置 24
4.13 触摸屏测试配置 25
4.14 按键测试配置 25
4.15 电源按键测试配置 26
4.16 Wi-Fi 测试配置 26
4.17 蓝牙测试配置 26
4.18 以太网测试配置 27
4.19 摄像头测试配置 27
4.20 闪光灯测试配置 28
4.21 音频模块测试配置 28
4.22 U 盘测试配置 29
4.23 USB3.0 测试配置 30
4.24 USB_HOST 测试配置 30
4.25 USB_OTG 测试配置 30
4.26 USB 数字耳机测试配置 30
4.27 HDMI 输出测试配置 31
4.28 CVBS 输出测试配置 31
4.29 界面配置 31

5 FAQ 34
5.1 如何添加新的板型方案 34

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2 如何调整测试用例在界面显示的顺序 34
5.3 为什么音频测试没有声音播放 34
5.4 无法识别触摸屏 34
5.5 如何解决串口打印是乱码 35

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文主要介绍全志平台的板卡测试系统 DragonBoard 及其使用方法，为用户对 DragonBoard 的使用和二次开发提供参考。

1.2 目标读者

DragonBoard 的开发/使用/维护人员。

1.3 适用范围

适用与 DragonBoard 配套的 linux-4.9、linux-5.4、linux-5.10、linux-5.15、linux-6.6 内核平台。目前支持的 IC 平台：

内核版本	IC 平台
linux-4.9	A100,A133,T509,T507-R
linux-5.4	A100,A133,H618,TV303
linux-5.10	A100,A133,T3,T3-C,A40I-H,A40I-C,T3PRO,T507-R
linux-5.15	A100,A133,A523,A527,T527,T736
linux-6.6	A333,A537,A733

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 系统介绍

2.1 系统简介

DragonBoard 是一个基于 Linux BSP，同时集成了 DirectFB 的图形化板卡测试系统。该系统旨在检测板卡能否在特定的环境下正常工作。

DragonBoard 固件支持 USB 方式烧录到板卡上，也支持卡启动。

DragonBoard 的测试项目分为两类：自动测试和手动测试。自动测试项包括内存、时钟、eMMC、NAND、UFS、Wi-Fi、蓝牙、重力传感器、光线传感器、GPS 等；手动测试项包括 PMU、触摸屏、SD 卡、U 盘、按键、霍尔传感器、振动马达、摄像头、闪光灯、音频模块等。

DragonBoard 在板卡上电开机后会自动加载运行测试用例，将测试项目显示到 LCD 屏上，用户可直观看出哪些测试项目通过、失败或待测状态。

DragonBoard 完成一张板卡测试所需要的时间具有一定的浮动性，主要受以下几个因素影响：

- 主频和 DDR 频率；
- 测试项的数量；
- 测试流水线和工人的熟练程度。

主频和 DDR 频率决定了板卡系统的运行速度，越高的频率测试程序运行得越快，所需的时间就越短；测试项的数量会影响系统负荷和工人操作时间；另外测试流水线设置是否合理以及工人的熟练程度也会直接影响测试时间。

2.2 源码结构介绍

DragonBoard 系统的源代码在 longan/test/dragonboard 目录下，具体的源码结构如下所示：

```
├── common
│   ├── adb
│   ├── rootfs
│   └── scripts
├── output
│   ├── bin
│   ├── data
│   ├── firmware
│   └── fonts
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
├──── rootfs        //根目录，是进入内核时的目录
│   ├──── autorun.sh
│   ├──── bin
│   ├──── dev
│   ├──── dragonboard
│   ├──── etc
│   ├──── home
│   ├──── init -> bin/busybox
│   ├──── lib
│   ├──── linuxrc -> bin/busybox
│   ├──── mnt
│   ├──── opt
│   ├──── proc
│   ├──── root
│   ├──── sbin
│   ├──── sys
│   ├──── tmp
│   ├──── usr
│   └──── var
├──── src                //源码目录
│   ├──── build.sh      //DragonBoard的编译工具
│   ├──── core          //主代码目录，包括测试用例的解析加载，视图的加载
│   ├──── include       //头文件
│   ├──── lib           //链接文件
│   ├──── Makefile
│   ├──── rule.mk
│   ├──── testcases     //测试用例目录，各个测试用例的代码
│   └──── view          //视图目录
├──── sysroot
│   ├──── bin
│   ├──── cedar-dev-pkg-test
│   ├──── dev
│   ├──── etc
│   ├──── include
│   ├──── lib
│   ├──── sbin
│   ├──── usr
│   └──── var
```

DragonBoard 系统支持的测试用例的配置文件在 longan/device/config/chips/{IC}/configs/{BOARD}/dragonboard 目录下，具体的目录结构如下所示：

```
├──── dragonboard
│   ├──── BoardConfig.mk
│   ├──── env.cfg
│   ├──── sys_partition.fex
│   └──── test_config.fex      //测试用例的参数配置文件
```

2.3 系统配置与编译

由于 DragonBoard 系统是基于 Linux BSP 编译的，因此需要待测试硬件平台的 Linux BSP，即 Linux 内核。编译系统默认使用 Android 系统的 Linux 内核。DragonBoard 系统的源代码位于 longan/test/dragonboard 目录下，由于 DragonBoard 是一个单独的板卡测试系统，需要单独编译。具体的配置与编译步骤如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.3.1 配置 DragonBoard

此处以 A100 平台的 b3 板型为例：

步骤 1：进入 longan 目录执行./build.sh config，选择 linux：

```
longan$ ./build.sh config

Welcome to mkscript setup progress
  All available platform:
    0. android
    1. linux
  Choice [linux]: 1
```

步骤 2：选择 dragonboard：

```
All available linux_dev:
  0. bsp
  1. dragonboard
  2. sata
  3. longan
  4. tinyos
Choice [bsp]: 1
```

步骤 3：选择 IC 为 a100：

```
All available ic:
  0. a100
  1. a133
  2. a50
  3. a63
  4. a64
  5. f113
  6. f133
  7. h3
  8. h6
  9. h616
  10. r328-s2
Choice [v459]: 0  //此处根据实际使用平台选择
```

步骤 4：选择 b3 板子：

```
All available board:
  0. b1
  1. b3
  2. fpga
  3. perf1
  4. perf2
  5. qa
  6. ver
Choice [b1]: 1  //此处根据实际板子型号选择
```

步骤 5：选择 flash，默认为 eMMC 选择 default 则可:

```
All available flash:
  0. default
  1. nor
Choice [default]: 0  //此处根据实际情况选择
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

步骤 6：选择内核版本:

```
All available kern_ver:
  0. linux-3.10
  1. linux-3.4
  2. linux-4.4
  3. linux-4.9
  4. linux-5.4
Choice [linux-4.9]: 3  //此处根据实际情况选择
```

2.3.2 编译 DragonBoard

执行完上述配置步骤后，就可以编译 DragonBoard 了，编译 DragonBoard 相当于编译 linux 最小系统 + linux 应用程序。进入 longan 目录执行./build.sh 则可。

```
longan$ ./build.sh
```

2.3.3 生成固件签名密钥

如果想要编译出非安全固件，则该步骤可跳过。

如果想要编译出安全固件，需要先在 longan/build 目录下执行指令生成固件签名密钥，具体操作如下：

```
longan/build$ ./createkeys

All valid Sunxi ic:
  0. a100
  1. a133
  2. a50
  3. a63
  4. a64
  5. f113
  6. f133
  7. h3
  8. h6
  9. h616
 10. r328-s2
Please select a ic[a133]: 1  //此处需要根据实际情况选择
```

说明

生成固件签名密钥一个 IC 平台只需运行一次即可。

2.3.4 打包 DragonBoard

执行完上述编译步骤后，就可以打包 DragonBoard 固件了。以下展示不同打包指令生成各种不同规格的固件：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
longan$ ./build.sh pack                  //uart0固件，可测TF卡
longan$ ./build.sh pack_debug            //card0固件，可查看串口log，但不能测TF卡
longan$ ./build.sh pack_secure           //安全uart0固件，能测TF卡
longan$ ./build.sh pack_debug_secure     //安全card0固件，可查看串口log，但不能测TF卡
```

打包执行完毕后，会在 longan/out 目录下生成固件，固件的名称分别对应如下所示：

```
a100_dragonboard_b3_uart0.img            //uart0打印非安固件
a100_dragonboard_b3_card0.img            //card0打印非安固件
a100_dragonboard_b3_uart0_secure_v0.img  //uart0打印安全固件
a100_dragonboard_b3_card0_secure_v0.img  //card0打印安全固件
```

2.3.5 烧写 DragonBoard

支持两种方式烧写 DragonBoard 固件，分别是：

1. eMMC/NAND/UFS 启动。使用 PhoenixSuit 或 PhoenixUSBPro 通过 USB 接口烧写固件。
2. SD 卡启动。使用 PhoenixCard 往 SD 卡烧写固件，制作“启动卡”。

2.3.6 编译及验证 Android & DragonBoard 二合一固件注意事项

2.3.6.1 编译 Android & DragonBoard 二合一固件

说明

编译 Android & DragonBoard 二合一固件需要注意区分 GRF 版本与非 GRF 版本，两种版本编译方法是不同的。

对于有编译 Android & DragonBoard 二合一固件需求的用户，可以修改 android/device/softwinner/common/misc/Android.mk 文件中 DEFAULT_BOOT_SLOT 的值来决定编译的二合一固件第一次开机时进入何种系统。

当设置 DEFAULT_BOOT_SLOT=A，则编出的二合一固件第一次开机进入 Android 系统；

当设置 DEFAULT_BOOT_SLOT=B，则编出的二合一固件第一次开机进入 DragonBoard 系统。

完成上述设置后，后续的操作步骤因 Android 系统源码是属于 GRF 版本或非 GRF 版本而有差异。

2.3.6.2 非 GRF 版本下编译 Android & DragonBoard 二合一固件

1. 编译 DragonBoard 系统 (操作同上 2.3 章节，可不用执行打包操作)；
2. 编译 Android 系统：

此处以 A133 平台的 b6 板型为例：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

步骤 1：进入 longan 目录执行./build.sh config，选择 android：

```
longan$ ./build.sh config
 Welcome to mkscript setup progress
  All available platform:
   0. android
   1. linux
  Choice [linux]: 0
```

步骤 2：选择 IC 为 a133：

```
  All available ic:
   0. a133
  Choice [a133]: 0  //此处根据实际使用平台选择
```

步骤 3：选择 b6 板子：

```
  All available board:
      0. ad86310vat
      1. ad86310vc
      2. b1
      3. b10
      4. b3
      5. b4
      6. b6
      7. b7
      8. evb
      9. fpga
      10. perf1
  Choice [b6]:6  //此处根据实际板子型号选择
```

步骤 4：选择 flash，默认选择 default 则可。

```
  All available flash:
   0. default
   1. nor
  Choice [default]: 0  //此处根据实际情况选择
```

步骤 5：选择内核版本：

```
  All available kern_ver:
   0. linux-5.4
  Choice [linux-5.4]: 0  //此处根据实际情况选择
```

步骤 6：选择 arch：

```
  All available arch:
   0. arm
   1. arm64
  Choice [arm64]: 1  //此处根据实际情况选择
```

步骤 7：编译 lichee：

执行完上述配置步骤后，就可以编译 lichee:

```
longan$ ./build.sh
```

步骤 8：为编译 Android 环境准备：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

此时需要切换到 longan 的上一层路径，即在 Android 路径下，执行导入环境变量、选择对应的方案。此处选择了 67。

```
longan$ cd ..
android12$ source build/envsetup.sh
android12$ lunch

  ...
  Lunch menu... pick a combo:
     1. ad86310vat-eng
     2. ad86310vat-user
     3. ad86310vat-userdebug
     4. ad86310vc-eng
     5. ad86310vc-user
  ...
     65. ceres_b6-eng
     66. ceres_b6-user
     67. ceres_b6-userdebug
     68. ceres_b7-eng
     69. ceres_b7-user
     70. ceres_b7-userdebug
  Which would you like? [aosp_arm-eng] :67
```

步骤 9：编译 Android：

```
android12$ make installclean
android12$ make -j16
```

3. 打包 Android 固件：以下展示使用不同打包指令生成各种不同规格的固件：

```
android12$ pack -d -t dragonboard          //card0打印二合一非安固件，不能测TF卡
android12$ pack -d -v -t dragonboard       //card0打印二合一安全固件，不能测TF卡
android12$ pack -t dragonboard             //uart0打印二合一非安固件，能测TF卡
android12$ pack -v -t dragonboard          //uart0打印二合一安全固件，能测TF卡
```

打包执行完毕后，会在 android/longan/out 目录下生成固件，固件的名称如下所示，以 A133 为例：

```
a133_android12_b6_card0.img                //card0打印非安固件
a133_android12_b6_card0_secure_v0.img      //card0打印安全固件
a133_android12_b6_uart0.img                //uart0打印非安固件
a133_android12_b6_uart0_secure_v0.img      //uart0打印安全固件
```

执行完上述操作之后得到的固件就是 Android & DragonBoard 二合一固件。

2.3.6.3 GRF 版本下编译 Android & DragonBoard 二合一固件

1. 编译 DragonBoard 系统（操作同上 2.3 章节执行编译，可不用执行打包操作）；
2. 编译 Android 系统：

此处以 A523 平台的 af86310vat 板型为例：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

步骤 1：为编译 Android 环境做准备：

此时需要切到 Android 主路径下，执行导入环境变量、选择对应的方案。此处选择了 2。

```
android14$ source build/envsetup.sh
android14$ lunch

Lunch menu .. Here are the common combinations:
   1. a523_af86310vat_arm64-user
   2. a523_af86310vat_arm64-userdebug
   3. a523_aiot_demo_arm_go-user
   4. a523_aiot_demo_arm_go-userdebug
   5. a523_evb_arm64-userdebug
   6. a523_evb_arm_go-user
   7. a523_evb_arm_go-userdebug
   8. a523_perf_arm_go-user
   9. a523_perf_arm_go-userdebug
   10. a523_pro_arm-user
   11. a523_pro_arm-userdebug
   ...
Which would you like? [aosp_arm-eng] : 2  //此处根据实际情况选择
```

步骤 2：编译 Android & dragonboard：

```
android14$ make -j16 -m ~/android_grf/android13/ --include-dragonboard
```

3. 打包 Android 固件：

以下展示使用不同打包指令生成各种不同规格的固件：

```
android14$ pack -d -t dragonboard          //card0打印二合一非安固件，不能测TF卡
android14$ pack -d -v -t dragonboard       //card0打印二合一安全固件，不能测TF卡
android14$ pack -t dragonboard             //uart0打印二合一非安固件，能测TF卡
android14$ pack -v -t dragonboard          //uart0打印二合一安全固件，能测TF卡
```

说明

打包指令中 pack -t 参数后最好带上 dragonboard。

打包执行完毕后，会在 android/longan/out 目录下生成固件，固件的名称如下所示，以 A523 为例：

```
a523_android14_af86310vat_card0.img              //card0打印非安固件
a523_android14_af86310vat_card0_secure_v0.img    //card0打印安全固件
a523_android14_af86310vat_uart0.img              //uart0打印非安固件
a523_android14_af86310vat_uart0_secure_v0.img    //uart0打印安全固件
```

执行完上述操作之后得到的固件就是 Android & DragonBoard 二合一固件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.3.6.4 验证 Android & DragonBoard 二合一固件

图 2-1: DragonBoard AllPass 按钮

上述如果设置了 DEFAULT_BOOT_SLOT=B，编译的二合一固件第一次开机默认会进入 DragonBoard 系统，如上图所示，且当自动测试项目全部都测试通过，会在 UI 界面自动弹出一个蓝色的 “AllPass” 按钮，待测试人员测试完所有手动测试项目后，点击这个 “AllPass” 按钮，之后重启设备就会进入 Android 系统。

说明

如果在 DragonBoard 界面上未点击过 “AllPass” 按钮，则设备重启之后仍会进入 DragonBoard 系统。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 测试流程

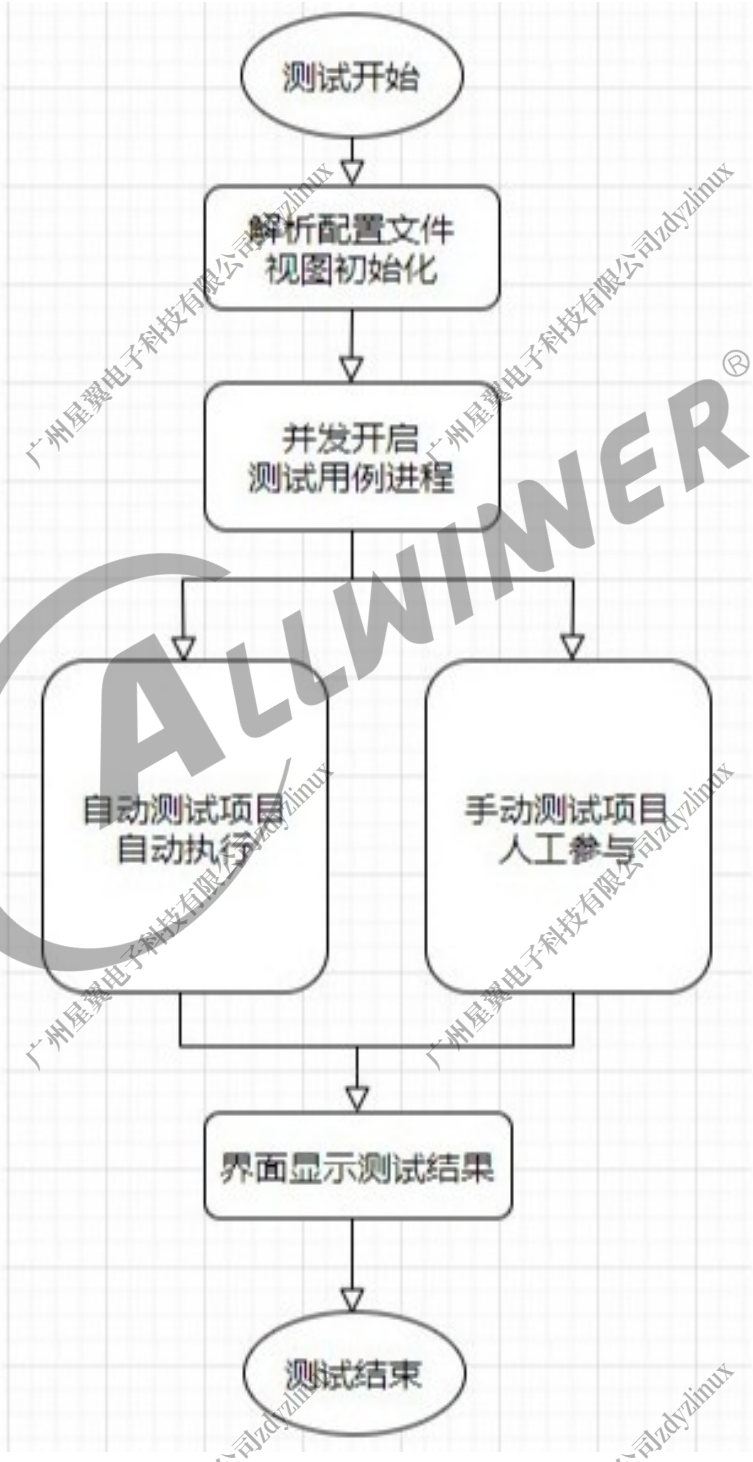

图 3-1: DragonBoard 测试流程图

版权所有 © 珠海全志科技股份有限公司。保留一切权利

DragonBoard 系统测试流程如上图所示，板卡上电开机后 DragonBoard 系统会自动运行，自动加载 test_config.fex 配置文件，LCD 屏显示主界面。配置的不同，主界面的布局也将不同。当呈现主界面，用户可直观看到各个测试项目的状态。

测试项目被分成两组：自动测试项和手动测试项。自动测试项的整个测试过程自动完成，无需用户干预。若测试通过，测试项目描成蓝色；若测试失败，测试项目描成红色。手动测试项需要用户参与。测试项目、Wi-Fi 模组搜索到的热点信息、LCD 显示色块的主界面如下图所示，如果没有配置某个测试项目开启或配置某设备的驱动错误，则不会显示相应的测试项目或该测试项目不能正常运行。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

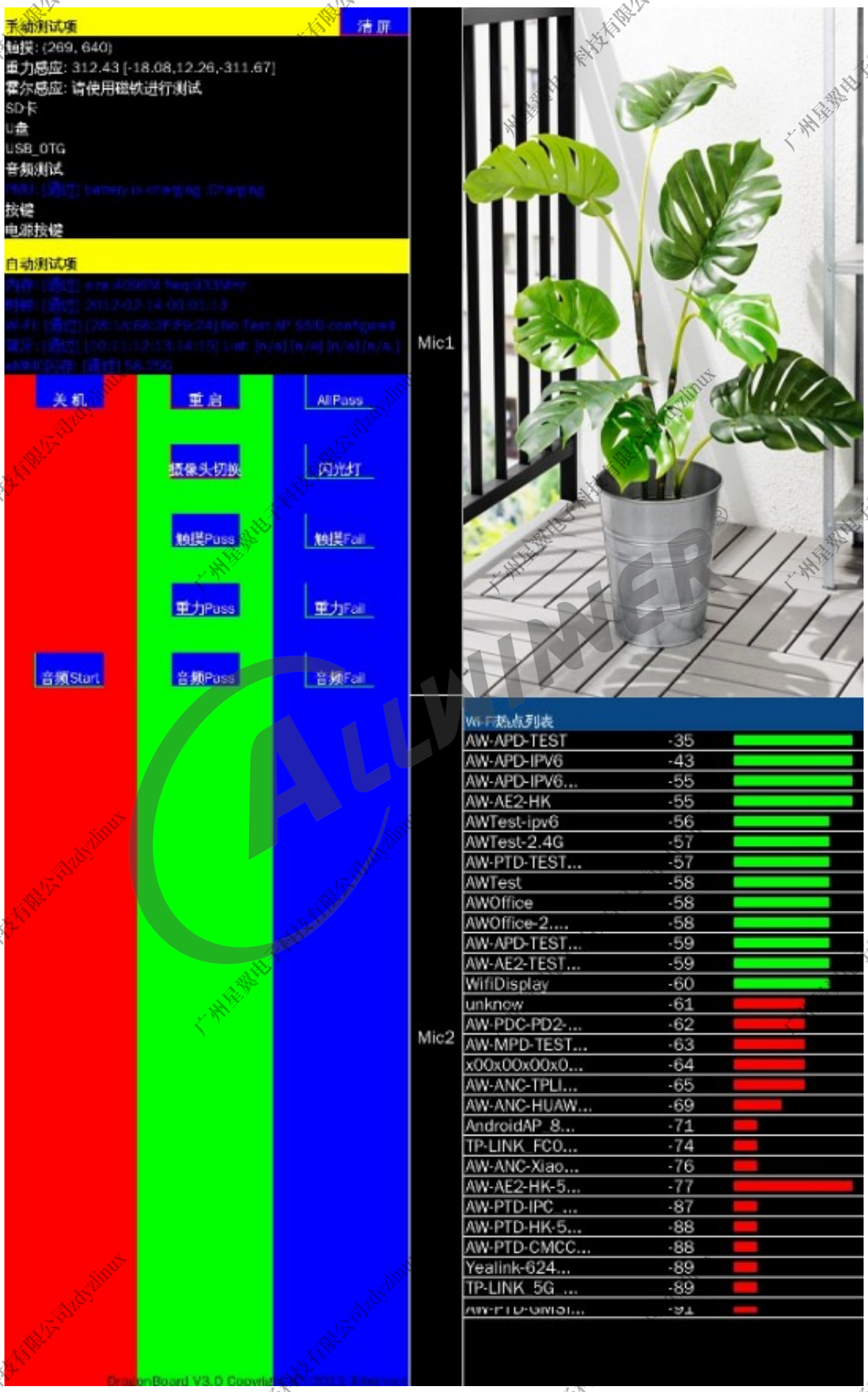

图 3-2. DragonBoard UI 界面

版权所有 © 2020 珠海全志科技股份有限公司。保留一切权利

说明

上图展示的一些模块，若硬件上没有，需要用户根据自己的原型机硬件修改 test_config.fex 配置文件。

3.1 PMU 测试

测试人员在设备开机后给设备插上电源适配器，则会自动进行 PMU 充电测试，充电功能正常该测试项目会显示 [通过]，反之显示 [失败]。

3.2 DRAM 测试

可配置 DragonBoard 开机自动运行 DRAM 内存测试，测试通过会显示内存容量和最高主频。

3.3 RTC 测试

可配置 DragonBoard 开机自动运行 RTC 时钟测试，测试通过会显示一个硬件 RTC 时间。

3.4 eMMC 闪存测试

可配置 DragonBoard 开机自动运行 eMMC 闪存测试，测试通过会显示 eMMC 闪存容量。

3.5 NAND 闪存测试

可配置 DragonBoard 开机自动运行 NAND 闪存测试，测试通过会显示 NAND 闪存容量。

3.6 UFS 闪存测试

可配置 DragonBoard 开机自动运行 UFS 闪存测试，测试通过会显示 UFS 闪存容量。

3.7 SD 卡测试

测试人员给板子插上一张 SD 卡或 TF 卡后等待几秒，如果测试通过该测试项目会显示 [通过]+ 容量，字体颜色描成蓝色，否则该测试项目显示 [失败]，字体颜色描成红色。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.8 重力传感器测试

当前重力传感器支持手动测试或自动测试。

1. 若配置为手动测试，则需测试人员判断三个方向的重力值是否符合通过标准，符合则点击界面上的 “重力 Pass” 按钮，反之则点击 “重力 Fail” 按钮。
2. 若配置为自动测试，则开机后若重力传感器有数据上报则显示 [通过]，反之显示 [失败]。

3.9 霍尔传感器测试

设备开机后，测试人员将磁铁在霍尔传感器附近移动，如果霍尔传感器工作正常，则该测试项目会显示 [通过]，反之显示 [失败]。

3.10 光线传感器测试

可配置 DragonBoard 开机自动运行光线传感器测试用例，如果光线传感器工作正常，则该测试项目会显示 [通过]，反之显示 [失败]。

3.11 GPS 测试

可配置 DragonBoard 开机自动运行 GPS 测试用例，如果 GPS 工作正常，则该测试项目会显示 [通过]，反之显示 [失败]。

说明

如果要在室内进行 GPS 传感器测试，需要在室内加装 GPS 信号放大器。

3.12 触摸屏测试

触摸屏测试支持：手动测试、半自动测试或自动测试三种测试方式。

1. 手动测试：手动测试需要测试人员通过手触摸屏幕，根据画线轨迹是否跟随手移动轨迹，人工判断是否 Pass，然后点击界面上的 “触摸 Pass” 按钮确认该测试项通过，该测试项会显示 [通过]；若点击界面上的 “触摸 Fail” 按钮确认该测试项不通过，则该测试项会显示 [失败]。
2. 半自动测试：半自动测试需要测试人员手动触摸屏幕，当测试用例检测到有触摸数据时，则自动判定该测试项 Pass。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3. 自动测试：自动测试是设备开机后自动加载 TP 驱动，测试用例若能够识别到预定义的 TP 设备名，则自动判定该测试项 Pass。

说明

上述半自动和自动测试方法，仅能测试触摸屏 IC 的硬件连通性是否正常，不能判断触摸画线功能是否完全正常。

3.13 按键测试

1. 若指定按键，比如在 test_config.fex 中定义了测试 “音量 +” 、“音量-” 按键，则当测试人员按下 “音量 +” 和 “音量-” 按键各一下，测试通过会显示 [通过]，反之显示 [失败]。
2. 若未指定按键，则按下任一按键，该测试项目均显示 [通过]。

3.14 电源按键测试

电源按键为手动测试，测试人员在设备开机 DragonBoard 系统运行起来后按下电源按键一下，测试通过会显示 [通过]，反之显示 [失败]。

3.15 Wi-Fi 测试

可配置 DragonBoard 开机自动运行 Wi-Fi 测试用例，扫描周围环境中的 Wi-Fi 热点，并把扫描到的热点显示到 LCD 屏幕界面上，热点按信号强度依次显示，同时也会显示出信号的 DB 值和信号强度能量条。默认信号强度大于-60db 的能量条为绿色，表示信号强度较好；信号强度小于-60db 的能量条为红色，表于信号强度较差。

Wi-Fi 功能测试正常则会显示 [通过]+MAC 地址，反之显示 [失败]。

说明

若当所有热点信号强度均低于-60db，说明 Wi-Fi 天线可能存在问题。

3.16 蓝牙测试

可配置 DragonBoard 开机自动运行蓝牙测试用例，扫描周围环境中可配对的蓝牙设备，若配对成功，则通过串口打印配对的蓝牙名称，该测试项目会显示 [通过]+MAC 地址，反之显示 [失败]。

说明

要确保蓝牙测试准确，需要特别注意两点：第一，确保附近有蓝牙设备打开。第二，确保该蓝牙设备设置的状态是：让附近所有的蓝牙设备均可检测。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.17 以太网测试

要确保从路由器端引出的网线是可以自动分配 IP 地址的，从路由器引出一根网线插入到待测设备的网口上，DragonBoard 会自动运行以太网测试用例，若测试成功该测试项目显示 [通过]，反之显示 [失败]。

3.18 摄像头测试

可配置 DragonBoard 开机自动运行摄像头测试用例，测试人员可在 LCD 屏幕右上角的方框内看到摄像头拍的预览照片，点击屏幕上的 “切换” 按钮可切换前、后置摄像头。

3.19 闪光灯测试

测试人员点击 LCD 屏幕上的 “闪光灯” 按钮即可控制摄像头处闪光灯的亮、灭。

3.20 音频模块测试

音频模块的测试支持两种测试方式。

音频模块测试方式一：

1. 系统开机启动后会立即播放一段测试音频，测试板载喇叭输出以及外接模拟耳机输出状态，此时可以通过外听喇叭输出确认喇叭输出是否正常或可以插上模拟耳机接听确认耳机输出是否正常。
2. 在开机测试音频播放完毕后会进入板载麦克风录音状态，录制周围环境声音，而屏幕中上方会有一个 MIC1 录音音量指示条用于检测确认板载麦克风是否能正常录音，如果 MIC1 音量指示条有幅度变化则表示板载麦克风录音正常，反之 MIC1 音量指示条没有幅度变化则表示板载麦克风没接或异常。
3. 当 2. 检测到 MIC1 录音强度达到一定等级的时候 (>= 1/3)，此时可以插入带麦模拟耳机进行耳机麦克风录音测试，录制周围环境声音，而屏幕中下方会有一个 MIC2 录音音量指示条用于检测确认插入带麦模拟耳机的耳机麦克风是否录音正常，如果 MIC2 音量指示条有幅度变化则表示耳机麦克风录音正常，反之 MIC2 音量指示条没有幅度变化则表示插入的不是带麦耳机或耳机麦克风异常。
4. 当 3. 检测到 MIC2 录音强度达到一定等级的时候 (>= 1/3)，音频测试项会显示 [通过]。

音频模块测试方式二：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1. 开机音乐：系统开机启动后会自动播放一段开机测试音频，音频播放默认支持喇叭输出和模拟耳机输出。
2. 音频测试开始：单次点击屏幕上的 “音频 Start” 按钮即可正式开始音频基础功能测试。
3. 音频测试结果判定：需要人工判断识别，若板载喇叭/模拟耳机能正常播放出声音且能够正常播放出录制的环境音，则表示音频器件皆良好，测试通过，此时可点击屏幕上的 “音频 Pass” 按钮，音频测试项会显示 [通过]，并停止音频测试；若喇叭/模拟耳机一直播放无声则表示喇叭/耳机器件异常；若播放板载 MIC 录制音频无声，则表示板载 MIC 器件异常；模拟耳机同理，此时可点击屏幕上的 “音频 Fail” 按钮，音频测试项会显示 [失败]，并结束音频测试。
4. 音频测试模式：音频测试开始后，会先播放一段测试提示音频 “录音测试开始”，随后会默认进行时长约 5s 的环境音录制，随后会继续播放一段测试提示音频 “播放测试开始” 并跟着播放之前进行录制的音频，如此不断循环重复，直至完成音频测试结果判定后停止测试循环。
5. 板载喇叭 + 板载 MIC 与四节模拟耳机 + 耳机 MIC 切换逻辑：在固定的音频测试模式下，在不插入模拟耳机时，默认固定音频输出从板载喇叭播放输出，默认输入从板载 MIC 录制输入；在插入四节带麦模拟耳机后，此时会自动切换音频输出为模拟耳机播放输出，默认输入会切换为耳机 MIC 录制输入，支持实时、反复切换。
6. 重新点击屏幕上的 “音频 Start” 可以再次触发音频测试开始，点击屏幕上的 “音频 Pass” 或 “音频 Fail” 可以刷新音频测试结果并停止音频测试。

说明

进行该项测试时，请确认所使用的耳机为模拟耳机，避免误用数字耳机。

3.21 U 盘测试

测试人员可在设备开机后插入 U 盘，如果 U 盘识别正常，则该测试项目会显示 [通过]+ 容量，反之显示 [失败]。

3.22 USB 3.0 测试

测试人员可在设备开机后在 USB3.0 接口插入 USB 3.0 的 U 盘，如果 U 盘识别正常，则该测试项目会显示 [通过]，反之显示 [失败]。

3.23 USB_HOST 测试

测试人员使用 USB 数据线将平板的 microUSB 接口连接到电脑上的 USB 接口，如果 USB 能够正常识别，USB 主机测试项目会显示 [通过]，反之显示 [失败]。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.24 USB_OTG 测试

测试人员可在设备开机后插入 U 盘，如果 U 盘识别正常，则 USB_OTG 测试项会显示 [通过]，反之显示 [失败]。

3.25 USB 数字耳机测试

测试人员可在设备开机后插入 USB 数字耳机，如果 USB 数字耳机识别正常，则该测试项会显示 [通过]，反之显示 [失败]。

说明

进行该项测试时，请确认所使用的 Type-C 接口的耳机为 USB 数字耳机，避免误用 USB 模拟耳机。

3.26 HDMI 输出测试

测试人员可在设备开机后从设备的 HDMI 输出口引出数据线连接至带 HDMI 输入接口的显示器，若显示器上能显示出多个色块（colorbar），则说明 HDMI 输出功能正常，该测试项会显示 [通过]，反之显示 [失败]。

3.27 CVBS 输出测试

测试人员可在设备开机后从设备的 CVBS 输出口引出数据线连接至带 CVBS 输入接口的显示器，若显示器上能显示出多个色块（colorbar），则说明 CVBS 输出功能正常，该测试项会显示 [通过]，反之显示 [失败]。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 配置文件

DragonBoard 提供了一个灵活的配置文件 test_config.fex，位于 longan/device/config/chips/{IC}/configs/{BOARD}/dragonboard 目录下，方便用户定制自己的测试项目，提高测试效率。

配置文件支持实现以下功能：

1. 开启或关闭某个测试用例；
2. 修改测试用例的参数；
3. 修改 LCD 屏显 UI 界面的语言和颜色。该脚本使用 ini 文件格式，由段、键和值三者组成，通常一个段表示一个模块的配置。该配置文件使用 UTF-8 编码。

4.1 模块配置示例

测试模块配置示例：

```
[example]
display_name= "Example"
activated  = 1
program    = "example.sh"
category   = 0
run_type   = 1
```

- example: 表示一个模块配置 example。
- display_name: 当前测试模块显示到界面的名称，字符串类型，最多可容纳 64 个字节。如果为空，测试程序不会运行。如需显示双引号，使用下面语法：

```
display_name = string:"Example"
```

如果模块显示的名称需要不同的语言支持，修改 display_name 字段即可。

- activated：0：不测试该模块；1：测试该模块。如果用户的方案不需要测试当前模块，请将该项置 0；否则，请将该项置 1。
- program：模块的测试用例可执行程序名称，可在 DragonBoard/src/testcases 目录找到，字符串类型，最多可容纳 16 个字节。
- category：0：自动测试模块；1：手动测试模块。
- run_type：0：等待当前模块的测试程序执行完毕再运行下一个模块的测试程序；1：不等待当前模块的测试程序执行完毕；一般为了提高整体的测试速度，对于耗时较长的测试程序建议填 0，反之填 1。注意，当 category = 1，即手动测试模块时，该项无效。因为手动测试模块会阻止其他测试模块加载，系统可能无法继续加载其他测试模块。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 PMU 测试配置

PMU 充电测试，具体配置如下所示：

```
[pmu]
display_name= "PMU"
activated  = 1
program    = "pmutester.sh"
category   = 1
run_type   = 1
```

4.3 DRAM 测试配置

DRAM 内存测试，具体配置如下所示：

```
[dram]
display_name= "内存"
activated  = 1
program    = "memtester.sh"
category   = 0
run_type   = 1
dram_size  = 2048
test_size  = 8
platform   = "sun50iw10p1"
```

- dram_size: 板卡上使用了多大的 DRAM，以 MB 为单位，DRAM 测试程序会去检测板卡上 DRAM 的实际容量，如果小于该配置容量，则说明贴片存在问题，测试不通过。
- test_size：使用多大的容量测试 DRAM 的性能，单位是 MB，默认使用 8M。实际测 DRAM 的性能，需要设定较大的容量，但是这样会耗时较长，因此需要用户折中选择一个合适的值。8M 大概可以在 30 秒以内完成 DRAM 性能测试。
- platform：板卡 IC 平台。

4.4 RTC 测试配置

RTC 时钟测试，具体配置如下所示：

```
[rtc]
display_name= "时钟"
activated  = 1
program    = "rtctester.sh"
category   = 0
run_type   = 1
set_date   = "021400002023.00"
```

- set_date：设置写入 RTC 的初始时间值，如果不设置默认会使用“021400002012.00”。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.5 eMMC 测试配置

如果闪存是 eMMC 方案，则可开启该项测试，否则请关闭。具体配置如下所示：

```
[emmc]
display_name= "eMMC闪存"
activated  = 1
program    = "emmctester.sh"
category   = 0
run_type   = 1
test_size  = 128
```

- test_size：测试 eMMC 闪存读写的大小，单位是 MB。

说明

1. 请务必保证待测 eMMC 闪存为 “白片” ，否则可能会因 MBR 不对应而导致测试失败。
2. 打开 eMMC 闪存测试时，务必关闭 NAND 闪存测试、UFS 闪存测试。

4.6 NAND 测试配置

如果闪存是 NAND 方案，则可开启该项测试，否则请关闭。具体配置如下所示：

```
[nand]
display_name= "NAND闪存"
activated  = 1
program    = "nandtester.sh"
category   = 0
run_type   = 1
module_path= "/system/vendor/modules/nand.ko"
test_size  = 16
```

- test_size：测试 NAND 闪存读写的大小，单位是 MB。

说明

1. 请务必保证待测 NAND 闪存为 “白片” ，否则可能会因 MBR 不对应而导致测试失败。
2. 打开 NAND 闪存测试，务必关闭 eMMC 闪存测试、UFS 闪存测试。

4.7 UFS 测试配置

如果闪存是 UFS 方案，则可开启该项测试，否则请关闭。具体配置如下所示：

```
[ufs]
display_name= "UFS闪存"
activated  = 1
program    = "ufstester.sh"
category   = 0
run_type   = 1
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

1. 请务必保证待测 UFS 闪存为 “白片”，否则可能会因 MBR 不对应而导致测试失败。
2. 打开 UFS 闪存测试，务必关闭 eMMC 闪存测试、NAND 闪存测试。

4.8 SD 卡测试配置

SD 卡测试，具体配置如下所示：

```
[mmc]
display_name= "SD卡"
activated  = 1
program    = "mmctester.sh"
category   = 1
run_type   = 1
```

4.9 重力传感器测试配置

重力传感器测试，具体配置如下所示：

```
[gsensor]
display_name= "重力传感器"
activated  = 1
program    = "gsensortester.sh"
category   = 0
run_type   = 1
module_count= 1
module1_path= "/system/vendor/modules/sc7a20.ko"
device_count= 1
device1_name = "sc7a20"
device1_convert = 32
```

- category：若需要手动测试 gsensor，则将该参数配置为 1，即会在测试界面上生成 gsensor 测试 Pass/Fail 按钮，需要手动点击按钮确认；若需要自动测试 gsensor，则将该项配置为 0，测试程序在检测到 gsensor 数据后，会自动显示 [通过]。
- module_count：重力传感器驱动的数量。
- module1_path：重力传感器的驱动路径。
- device1_name：重力传感器的设备名。
- device1_convert：重力传感器转换值，转换值为标准重力值/convert，如：g 的转换值 = g 原始值 x 标准重力值/convert，若不配置，则默认为 1，即为不转换，显示原始值。

4.10 霍尔传感器测试配置

霍尔传感器测试，具体配置如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[hallsensor]
display_name= "霍尔传感器"
activated  = 1
program    = "hallsensor.sh"
category   = 1
run_type   = 1
module_count= 1
module1_path= "/system/vendor/modules/hall_sensor.ko"
device_count= 1
device1_name = "MH248"
```

4.11 光线传感器测试配置

光线传感器测试，具体配置如下所示：

```
[als]
display_name= "光线传感器"
activated  = 1
program    = "alstester.sh"
category   = 0
run_type   = 1
module_count= 1
module1_path= "/system/vendor/modules/stk3x1x.ko"
device_count= 1
device1_name = "stk3x1x_ls"
```

4.12 GPS 测试配置

GPS 测试，具体配置如下所示：

```
[gps]
display_name= "GPS"
activated  = 1
program    = "gpstester"
category   = 0
run_type   = 1
baud_rate  = 115200
device_node = "/dev/ttyS3"
pass_condition = "location"
```

- baud_rate：GPS 模块的通讯波特率。
- device_node：GPS 模块与设备数据传输使用的设备接口。
- pass_condition：设置 GPS 测试用例判定通过的条件，取值可以为 “location”、“time”、“data”。

location：表示需要获取到有效的定位信息（状态为’A’）才视为通过；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
time：表示需要获取到有效的时间信息（年份>2000且月份和日期>0）才视为通过；

data：表示只要接收到GPS数据即视为通过。
```

4.13 触摸屏测试配置

TP 触摸屏测试，具体配置如下所示：

```
;-----------------------------------------------------------------------------
;type(may unuesd):
;   0: DTP:
;   1: CTP
;
;tp_test_type(default 1):
;   depends on category=1
;   1:manual test,need press pass or fail button;
;   2:semi-automatic test,when get touch event will pass;
;-----------------------------------------------------------------------------
[tp]
display_name= "触摸"
activated  = 1
type      = 1
program    = "tptester.sh"
category   = 1
tp_test_type= 1
run_type   = 1
module_path = "/system/vendor/modules/gslX680new.ko"
device_name = "gslX680"
```

- type：触摸屏的类型，0 表示电阻屏，1 表示电容屏。电阻屏在系统启动后会运行 TP 校准的程序，只有通过调屏程序才会进入 DragonBoard 界面。
- category：测试类型：1 为手动测试，0 为自动化测试。
- tp_test_type：手动测试时测试方式选择，只有当 category=1 时才生效：1：手动测试，需要手动点击 “触摸 Pass” 或 “触摸 Fail” 按钮来确认测试结果；2：半自动测试，启动后用手触摸屏幕或画线，测试用例若能获取到触摸事件则该测试用例通过，反之失败。
- module_path：触摸屏的驱动全路径。

4.14 按键测试配置

按键测试，具体配置如下所示：

```
[key]
display_name= "按键"
activated  = 1
program    = "keytester"
category   = 1
run_type   = 1
key_num    = 2
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
key_code1  = 114
key_code2  = 115
```

- key_num：选配项，若配置了 key_num，则表示为指定按键测试。如上述 key_code1 和 key_code2 分别为测试 “音量 +”、“音量-” 按键的键值。如不配置，则按下任一按键，该测试项会显示 [通过]。
- key_code：按键的键值。

4.15 电源按键测试配置

电源按键测试，具体配置如下所示：

```
[power_key]
display_name= "电源按键"
activated  = 1
program    = "pwkeytester.sh"
category   = 1
run_type   = 1
```

4.16 Wi-Fi 测试配置

Wi-Fi 测试，具体配置如下所示：

```
display_name= "Wi-Fi"
activated  = 1
program    = "wifitester.sh"
category   = 0
run_type   = 1
wifi_vnd   =
module_count= 1
module_path = "/system/vendor/modules/8723cs.ko"
module_args =
signal_low_threshold = -60
```

- wifi_vnd：为 common 时表示使用硬件自动检测功能，无需配置 module_count、module_path、module_args。
- module_path：需要加载的模块驱动全路径。
- module_args：模块的参数（可选）。
- signal_low_threshold：设定判定信号强度强弱的一个阈值，此处可为空，默认为-60。

4.17 蓝牙测试配置

蓝牙测试，具体配置如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[bluetooth]
display_name= "蓝牙"
activated  = 1
program    = "bttester.sh"
category   = 0
run_type   = 1
device_node = "/dev/ttyS1"
baud_rate  = 1500000
test_time  = 20
dst_bt     = ":"
bt_vnd     = "realtek"
module_path =
```

- device_node：蓝牙模块的通讯串口号。
- baud_rate：蓝牙模块的通讯波特率。
- test_time：蓝牙模块搜索信号的次数。
- dst_bt：设备地址过滤列表，设置为冒号表示不过滤。
- bt_vnd：为 common 时表示使用硬件自动检测功能，无需配置 module_path 参数。

4.18 以太网测试配置

以太网测试，具体配置如下所示：

```
[ethernet]
display_name= "以太网"
activated  = 1
program    = "ethtester.sh"
category   = 0
run_type   = 1
module_path =
module_args =
```

4.19 摄像头测试配置

摄像头测试，具体配置如下所示：

```
[camera]
activated  = 1
module_count= 8
module1_path= "/system/vendor/modules/videobuf2-common.ko"
module2_path= "/system/vendor/modules/videobuf2-memops.ko"
module3_path= "/system/vendor/modules/videobuf2-dma-contig.ko"
module4_path= "/system/vendor/modules/videobuf2-v4l2.ko"
module5_path= "/system/vendor/modules/vin_io.ko"
module6_path= "/system/vendor/modules/nvp6158.ko"
module7_path= "/system/vendor/modules/rn6854m_mipi.ko"
module8_path= "/system/vendor/modules/vin_v4l2.ko"
dev_cnt    = 2
csi_cnt    = 2
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
fps       = 25
video_cnt  = 6
video_id0  = 0
video_fps0 = 30
video_id1  = 1
video_fps1 = 30
video_id2  = 2
video_fps2 = 30
video_id3  = 3
video_fps3 = 30
video_id4  = 4
video_fps4 = 30
video_id5  = 5
video_fps5 = 30
```

- module_count：需要加载的驱动数量。
- module_path：驱动全路径。
- csi_cnt：模组数量。
- video_cnt：video 节点个数，适用于多个摄像头验证。
- video_id0：video 节点 id。
- video_fps0：video 节点帧率。

说明

摄像头测试用例：只关心是否能出图，不关心图像角度。

4.20 闪光灯测试配置

摄像头闪光灯的测试，具体配置如下所示：

```
[flashlight]
activated  = 1
sensor_name = "imx319_mipi"
```

- sensor_name：摄像头模组的名称。

说明

由于闪光灯属于摄像头模组上的元件，使用的是摄像头模组的驱动，进行闪光灯测试前最好先进行摄像头的测试。

4.21 音频模块测试配置

音频模块测试，具体配置如下所示：

```
[mic]
display_name= "音频测试"
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
activated  = 1
program    = "mictester"
category   = 1
run_type   = 1
delay      = 3
volume     = 0
music_playtime = 20
music_file = "/dragonboard/data/test48000.pcm"
samplerate = 48000
mic1_used  = 1
mic2_used  = 1
mic1_gain  = 1
mic2_gain  = 1
mic1_threshold =4096
mic2_threshold =4096

[soundcard]
card       = 0
device     = 0
mic_spk_test = 1
test_wav_time = 5
;only used for singal codec output and default LINEOUTR/HPOUTR output.
dac_swap   = 0
;only used to dual Speaker L channel and R chennel test, singal Speaker should not use.
dual_spk_channel_test = 0
test_version = 1
play_card_name = "audiocodec"
cap_card_name = "ahubi2s0"
auto_start = 1
```

- program：对于 A 系列平台配置成 “mictester”，对于 T 系列平台配置成 “soundtest.sh”。
- mic_spk_test：用于选择当前音频测试方式，当值为 0 时，则选用音频测试方式一；当值为 1 时，则选用音频测试方式二。
- test_wav_time：当选用音频测试方式二时有效，用于控制进入录音并播放测试循环后录音与播放的音频时长，当前默认时长为 5s。
- dac_swap：当外围硬件电路仅支持单喇叭输出且默认从右声道喇叭 (HPOUTR/LINEOUTR) 输出时需要将该配置打开，喇叭才能正常出声。
- dual_spk_channel_test：当外围硬件电路支持左 + 右声道双喇叭输出时，需要将该项配置打开，则可以额外支持左/右喇叭测试。
- test_version：如果播音和录音使用不同声卡，参数设置成 1。使用同一声卡设置成 0 或者不设置该参数。
- play_card_name：播音声卡设备名。
- cap_card_name：录音声卡设备名。
- auto_start：配置成 1 开机自动运行该用例，此时需将 category 配置成 0。

4.22 U 盘测试配置

U 盘测试，具体配置如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[udisk]
display_name= "U盘"
activated  = 1
program    = "udisktester.sh"
category   = 1
run_type   = 1
```

4.23 USB3.0 测试配置

USB3.0 测试配置，示例如下：

```
[usb3]
display_name= "USB 3.0"
activated  = 1
program    = "usb3tester.sh"
category   = 1
run_type   = 1
```

4.24 USB_HOST 测试配置

USB_HOST 测试，具体配置如下所示：

```
[Usb_HOST1]
display_name= "USB_HOST1"
activated  = 1
program    = "host1tester.sh"
category   = 1
run_type   = 1
```

4.25 USB_OTG 测试配置

USB_OTG 测试，具体配置如下所示：

```
[Usb_OTG]
display_name= "USB_OTG"
activated  = 1
program    = "otgtester.sh"
category   = 1
run_type   = 1
```

4.26 USB 数字耳机测试配置

USB 数字耳机测试，具体配置如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[USB_DIG_JACK]
display_name= "USB数字耳机"
activated  = 1
program    = "usbdigjack.sh"
category   = 1
run_type   = 1
```

4.27 HDMI 输出测试配置

HDMI 输出测试，具体配置如下所示：

```
[hdmi]
display_name= "HDMI"
activated  = 1
program    = "hdmitester.sh"
category   = 1
run_type   = 1
sound_file = "/dragonboard/data/test48000.pcm"
samplerate = 48000
```

4.28 CVBS 输出测试配置

CVBS 输出测试，具体配置如下所示：

```
[tv]
display_name= "TV"
activated  = 1
program    = "tvtester.sh"
category   = 1
run_type   = 1
sound_file = "/dragonboard/data/test48000.pcm"
samplerate = 48000
```

4.29 界面配置

显示界面的一些配置参数，具体配置如下所示：

```
[df_view]
manual_menu_name   = "手动测试项"
auto_menu_name     = "自动测试项"
clear_button_name  = "清屏"
Wi-Fi_menu_name    = Wi-Fi热点列表"
font_size          = 48
menu_bgcolor       = 1
menu_fgcolor       = 7
item_init_bgcolor  = 7
item_init_fgcolor  = 0
item_ok_bgcolor    = 7
item_ok_fgcolor    = 2
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
item_fail_bgcolor  = 7
item_fail_fgcolor  = 5
tp_draw_color      = 0
pass_str           = "通过"
fail_str           = "失败"
tv_scale_factor    = 95
```

- manual_menu_name：手动测试项的菜单显示内容。
- auto_menu_name：自动测试项的菜单显示内容。
- clear_button_name：清屏按键显示内容。
- font_size：测试用例名称的字体大小。
- menu_bgcolor：菜单的背景色，从 Color Index 中选择。
- menu_fgcolor：菜单的前景色，从 Color Index 中选择。
- item_init_bgcolor：测试项初始背景色，从 Color Index 中选择。
- item_init_fgcolor：测试项初始前景色，从 Color Index 中选择。
- item_ok_bgcolor：测试项通过背景色，从 Color Index 中选择。
- item_ok_fgcolor：测试项通过前景色，从 Color Index 中选择。
- item_fail_bgcolor：测试项失败背景色，从 Color Index 中选择。
- item_fail_fgcolor：测试项失败前景色，从 Color Index 中选择。
- tp_draw_color：触摸轨迹前景色，从 Color Index 中选择。为了能够清楚看到触摸轨迹，请选择和大部分区域的背景色相反的颜色。
- pass_str：测试项通过提示语。
- fail_str：测试项失败提示语。
- tv_scale_factor：HDMI 输出缩放因子。现在市场上的许多电视机都会对输入的视频裁边，导致视频内容不能完全显示或电视画面四周有黑边。针对这种情况，DragonBoard 会根据缩放因子对输出视频进行缩放。如：tv_scale-factor=80，配置输出为 1280x720 分辨率时，实际可显示的图像区域为 (1280x80%)x(720x80%)，即 1024x576。此缩放因子有效范围为 50~100，超出此范围将自动认为不缩放，也就是 tv_scale-factor=100。

Color Index 的选择如下所示：

Color	Index
白色	0
黄色	1
绿色	2
青色	3
洋红	4

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Color	Index
红色	5
蓝色	6
黑色	7

广州星翼电子科技有限公司zdyzlinux

5 FAQ

5.1 如何添加新的板型方案

在 longan/device/config/chips/{IC}/configs 路径下添加新的板型方案，可以先从其他板型的相应路径下拷贝一份板型方案，重命名后作为新的板型方案目录，将对应 Android 方案的 sys_config.fex 拷贝到新的方案目录。针对新的方案，修改 test_config.fex 文件。一般需要修改开启或关闭某些测试项目、模块的驱动、参数、在界面上显示的名称。

下面列举几个经常可能修改的模块：

1. Wi-Fi 模块：修改模块的驱动和参数；
2. G-Sensor 模块：修改模块的驱动和参数；
3. TP 模块：修改模块的驱动和参数；
4. Camera 模块：修改模块的驱动。注意模块数量改变后要修改 module_count。

5.2 如何调整测试用例在界面显示的顺序

如果想要调整测试用例在界面排布显示的顺序，调整 test_config.fex 配置文件中测试用例模块的顺序即可。

5.3 为什么音频测试没有声音播放

检查板级目录下的 board.dts 的 audio 配置项有没有打开。

检查内核下的设备树配置文件中 audio 配置项有没有打开。

同时，DragonBoard 的声音只可以从耳机或者外放输出，两者不可以同时输出，当外放没有声音时，需确认耳机是否有插入。

5.4 无法识别触摸屏

若系统启动后 LCD 显示屏有 UI 界面显示，但触摸屏幕画线无反应。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1. 若触摸测试项显示 [失败]，则说明驱动加载失败或未加载，请确认驱动是否有编译，test_config.fex 中配置的 TP 驱动路径是否正确；
2. 若触摸测试项无异常，但触摸无反应，则应检查排线等是否安装牢固。

5.5 如何解决串口打印是乱码

针对 A333、A537、A733 平台，升级了 DragonBoard 的串口波特率，支持 1.5M 波特率，若用户使用 115200 波特率发现串口打印为乱码，将波特率更换为 1500000 即可正常打印。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。